JAVA Lecture - 6. Date - 03/03/26

JAVA operator | Arithmetic + Unary + Logical + Bitwise + Assignment operators |.

operators : → 1. Arithmetic operators.
(+, -, *, ÷), %.
+=, -=, *=, /=, %=
(++, --) increament, decreament

```
Public class Demo
  Public static Void main (String [] args) {
   int a=5;
   int b= 10;
   int c = a+b;
   int d = a-b;
   int e = a*b;
   int f = b/a;
   int g = b% a;
  System.out.Println (c+",,"+d+",,"+e+",,"+f",,"+g);
   int h = a+2; // ~~this~~ Since this is okay in CSE
   h+=2;// h=h+2; //h=9.
   h-=2; // h=h-2; //7
   h*=3; // h=h*3;
   h/=5; // h=h/5;
   h%=// h=h%5;
  System.out.Println (h);
 }
}.
```

increament

```
int i = 6;
i++ // i=i+1;
System.out Print\n(i);
i--; i=i-1 i=6.
```

Preincreament

```
j=7;
~~j++~~;
```

```
~~++j~~; // --j;
```

Postfix

Relational Operator : ==, != <, >, <= >=
Boolean value,

c = (a != b) expressions.

Bitwise Operator → Bit manupulation,
&, |, ^, ~, >>, <<, >>>, &=, |=, ^=, >>= <<=, >>>=

A	B	A & B	A | B	A ^ B	~A	~B
0	0	0	0	0	1	1
0	1	0	1	1	1	0
1	0	0	1	1	0	1
1	1	1	1	0	0	0

Truth Table

left shift (<<)
Byte b = 8;
b = b<<1;

integer.

128 64 32 16 8 4 2 1
0 0 0 0 1 0 0 0 = 8
0 0 0 1 0 0 0 0 = 16

b = b<<1; 0 0 1 0 0 0 0 0 = 32

b = b<<1; 0 1 0 0 0 0 0 0 = 64

b = b<<1; 1 0 0 0 0 0 0 0 = -128

1's Compliment

1 0 0 0 0 0 0 0
0 1 1 1 1 1 1 1

2's Compliment

+ 1

1 0 0 0 0 0 0 0 = [128]

8 bit → 6th bit (+ve)
↳ 7th bit (-ve)
n bit → (n-2) bit (+ve)
↳ (n-1) bit (-ve)

b = b<<1 = 0 0 0 0 0 0 0 0 = 0

b = b<<1 = 0 0 0 0 0 0 0 0 = 0,

Type promotion → byte, short

byte b = 1;
b = b << 7
⇓
[b = (byte) b << 7]

[int]

byte = 5;
b = b * 5;
b = (byte) b * 5;

0 0 0 0 0 0 0 1

b = b << 7 = 1 0 0 0 0 0 0 0 = -128

b = b << 1 = 1 [0 0 0 0 0 0 0 0] → 0

[byte]

int i = b << 1 → 1, 1, 1, 1 ... 1 (0 0 0 0 0 0 0 0) = -256
24 — 8 bits

MSB

int i = 1; [0 0 0 0 0 0 0 0] [. . . . .] [. . . . .] [0 0 0 0 0 0 0 1]
i << 30 — 31st bit — 0th Bit — LSB

i << 1 [1, 0 . . . . . 0 . . . . . 0 . . . . . 0]
0th Bit

→ (-ve)

[i << 31] Integer . MIN_VALUE . (-∞)

[i << 32] → [Not Applicable]
[i << 33] →

i << 31 = i << (31 % 32)

[i << 32 ≈ i << 0 ≈ i] = 0 ✓ [i << 31 = i << (31 % 32)] = 0

- Right Shift →

Byte b = 8 — 0 0 0 0 1 0 0 0 = 8

~~b = (byte) b >>~~

b = (byte) b >> 1 ⇒ 0 0 0 0 0 1 0 0 = 4

byte b = -128 — 1 0 0 0 0 0 0 0 ⇒ -128

[b >> 1] — 1 1 0 0 0 0 0 0 ⇒ ⇒ -64 ✓

MSB — LSB

+/- 32 [int, long] → Actual Shifting

byte, short → Truncated value

>>> Shift Right with zero. → Dumb shift

byte b = -128 msb 1 0 0 0 0 0 0 0 lsb → -128

b >>> 1 0 1 0 0 0 0 0 0 ⇒ +64

Logical Operators → Expression

→ && (AND) & ;~

→ || (OR) |

```
int a = 5;
int b = 10;
int c = 15;
```

```
boolean d = (a<b) && (a<c);
```

T, f = T, f T, f

```
boolean e = (a<b) || (a<c);
```

= T	=	T	, T
T	=	f	, T
f	=	f	, f

Short-Circuit → &&

A || B
f
T
T
T

A || B. (True)
↓
T, B Not check

A	B	A && B
f	f	f
f	T	f
T	F	f
T	T	T

(False)

A && B
↓
f Not check B

Bitwise operator

& |

```
int a = 5;
    b = 10;
    c = 15
```

```
d = (a<b) & (a<c)
```

Does Not do short circuit

int n = 0 ; 1 ; 2

Assignment Operator →

int a = 5

Assignment operator

int a = 5
int b = 10

Boolean → a == b. → Relational Operator.

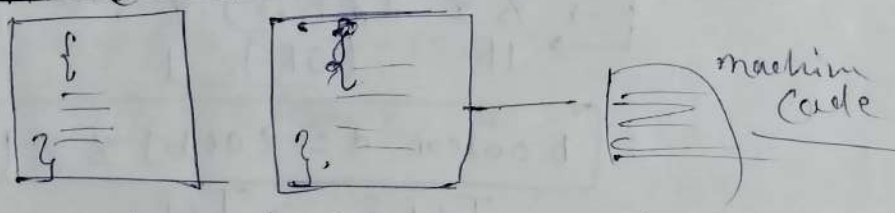
Special Operator → Ternary Operator →

operator predence → int c = (a*b+d-f/g << 2)

( ) → highest predence. b+(c*d) , (b+c)*d

Lecture 07 - JAVA Conditional Statement 04/03/2021

flow of control



- selection
- iteration
- jump

if statement if else, switch, loops.

JAVA LOOPS & JUMP Statements for, while, do while, Break, Continue Day - 08 Date. 8/03/26

Loops. 1 → while 2. do while 3. for.

~~do~~ while loop.

```
while (Boolean value) {→ True. false
  // do something.
  ~~sty~~ system.out.Println(1);
}
while (i<=10) {
  System.out.Print-ln (1);
  i++;
}
}}
```

```
do {
  //do something
} while expression
```

(i++<10)

```
do {System.out.Println(i
  i++;
} while (i<=10);
}}
```

//manu item selection

→ Do while.

1. Play game
2. Return saved game
3. exit.

~~for loop.~~

```
~~for (-) {~~ → int i=0 (i=1;) {
// do something
}}.
for (int i=1; i<=10; i++){
  System.out.Print-ln (i);
}
}}
```

→ flow of control of for

1. first assignment statement is execute (variable definition).
2. Then second conditional Statement is evaluated. (True/false)
4. Once loop body is finished, Control flow will go back to the for statement and third increament statement will be evaluate.
5. again Conditional statement is evaluated.
6. Repeat 2 - 5.

```
for (int i=1; i<=5; i++){
  for (int j=1; j<=i; j++){
    System.out.Print("*");
  }
  System.out.Println();
}}
```

Break

```
for (int i=1; i<=10; i++){
  System.out.Print-ln(i);
  if (i>5){
    break;
  }}
}
```

```
// whether a number is prime or not.
int p = 9;
// 2, 3, 4, 5 . . . 8   int i;
for( int i = 2; i<p; i++) {
    if (p % i == 0) {
    System.out.Println("The number is not Prime");
    Break;
}
}
else { if (i == p) {
    System.out.Println(" The number is prime");
```

Continue.

```
for (int i=1; i<=10; i++) {
    if (i % 2 != 0) {
    System.out.Println(i);
}}
```

```
for (int i = 1; i<=10; i++){
    if (i % 2 == 0) {
        Continue;
    }
    System.out.Println(i);
}}
```

Break in Nested loop.

```
for (int i=1; i<=10; i++){
  for (int j=1; j<=i; j++) {
    System.out.Print("*");
    if (j>=5) {
        break;
  }}
  System.out.Println();
}
```

```
outer:
    inner;
continue / break outer;
```

```
*
* *
* * *
* * * *
* * * * *
```

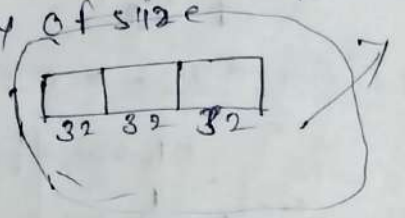
```
*
* *
* * *
* * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

Introduction to arrays | Declaration, Initialization, Traversal | String intro | Java Day-09.

Array → Collection of a data types.

```
int [] rollnums = new int [3]
```

- int → data types.
- rollnums → identifires. name
- new → special. Keyword. Allocation memory in Heap.
- [3] → Array of size



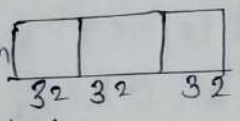
Memory

Contiques.

```
int [] rollnum :
rollnum = new int [3];
```

r1 → 101
r2 → 102
r3 → 103

```
int [] rollnum = new int [3]
```




```
rollnums [0] = 101;
rollnums [1] = 102;
rollnums [2] = 103;
System.out.println (rollnum [0] //101
                    (rollnum [1]//102
                    (rollnum [2]//103
```

```
int [] roll no = new int [3];
int x = 101;
for (int i = 0; i<3; i++) {
  rollnum [i] = x;
  x++;
}
for (int i = 0; i<3; i++) {
  System.out.Println (rollnums [i] );
```

---

rollnums

101	102	103
0	1	2

Size of Array

```
System.out Print.In (rollnums.length);
for i = 0; i<rollnums.length; i++) {
    // Array ka size mil jata Hain
}
```

roll Nums.

101	102	103
0	1	2

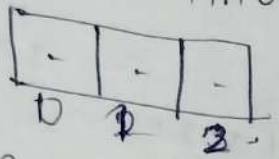
System.out.Print.In (rollnum [2]; rollnum[4]);

Index out of Bound Exception

Index Out of Bound Exception. : | java ka tarika batane ka Array limit out of Bound. |

Multi-Demensial Arrays. →

int [] rollnums. = new int [3] // Har eliment integer.

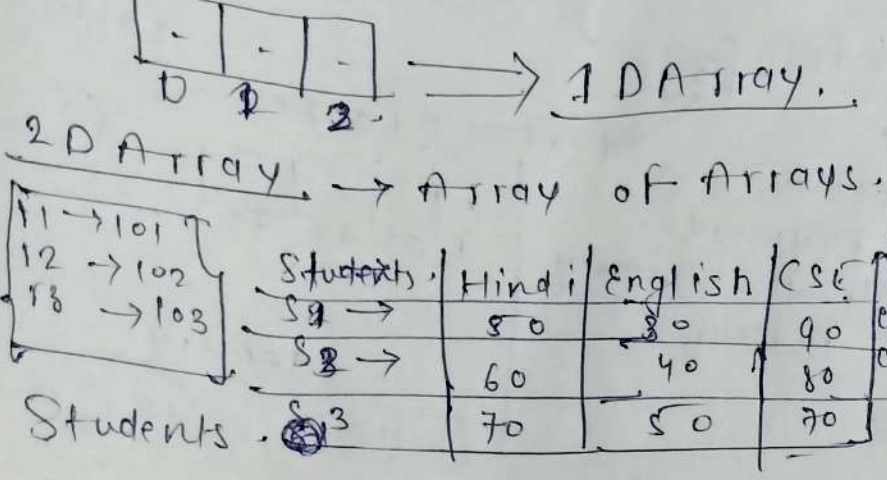


→ 1D Array.

Array of int

| conceptual Representation of 2D Array |

2D Array → Array of Arrays.



~~int student m~~ int [] [] marks = new int [3] [3];

← Marks →

Nested loop →

No of col

50	30	40	S1
60	40	80	S2
70	50	70	S3.

Eng H CSE (3x3)

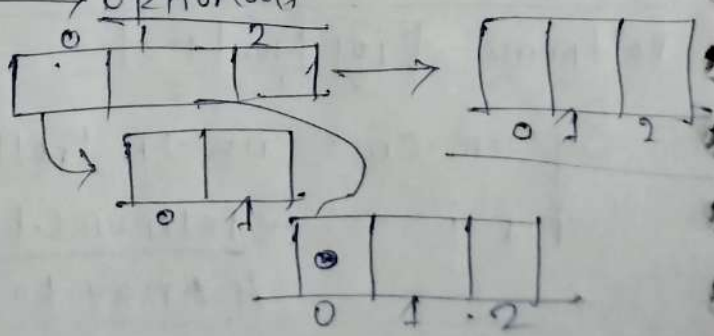
```
for (int row = 0; row < 3; row++) {
    for (int col = 0; col < 3; col++) {
        System.out.Print (marks. [row [col]];
    }
    System.out.Println ();
}
```

```
for (int row = 0; row < marks.length; row++) {
    for (int col = 0; col < marks[row].length; col++) {
        System.out.Print (marks[row] [col]);
    }
    System.out.Println ();
}
```

int [] [] marks = new int [3] [ ];

→ Optional

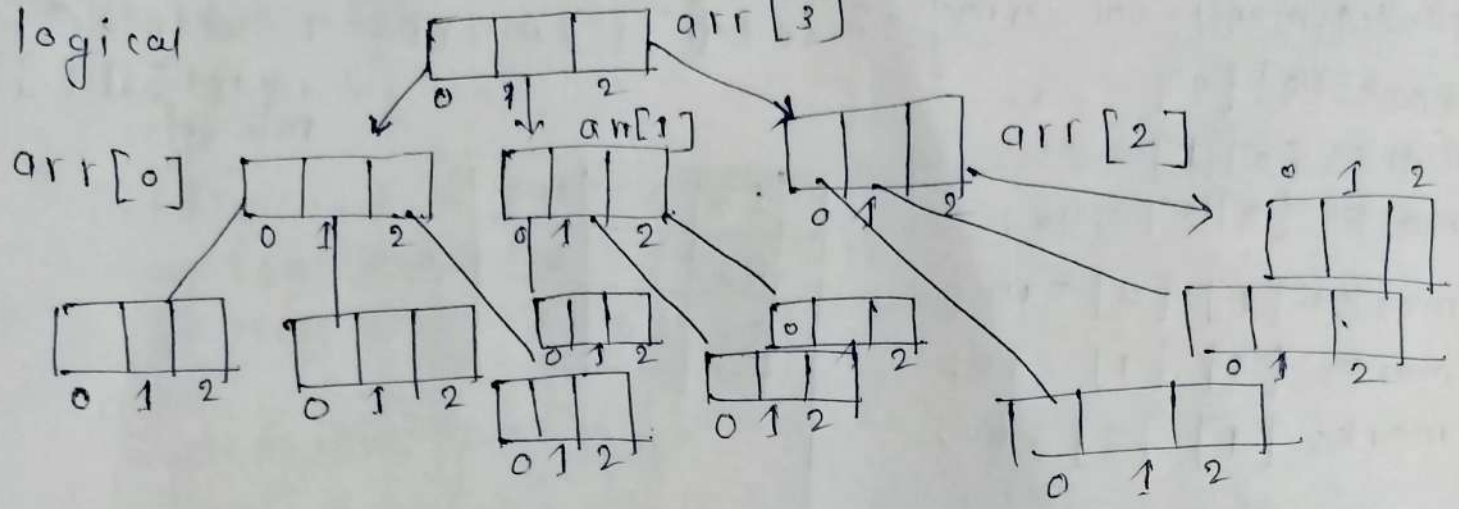
Marks [0] = new int [2];
Marks [1] = new int [3];
Marks [2] = new int [3];



3.D Array :

int [ ] [ ] [ ] arr = new int [3] [ ] [ ];

logical



1D.

int [ ] rollnums = new int [3]

rollno [0] = 4;
rollno [1] = 5;
⋮

~~rollnums [0]~~

int [ ] rollnums = {, 4, 5, 6}

2D. int [ ] marks = {
→ {4, 5, 6, 3}
→ {7, 8, 9}
→ {23, 34, 93}
};

/ 09.

```
Public class Day09.
Public Static Void main (String [] args) {
//Multidimensional arrays: int [] [] marks = new int [] []
                                                int [] [];
                                                row cols.
marks [0] [0] = 23;
marks [0] [1] = 25;
marks [0] [2] = 40;
marks [1] [0] = 40;
marks [1] [1] = 30;
marks [1] [2] = 60;
marks [2] [0] = 35
marks [2] [1] = 30
marks [2] [2] = 18;
```

23	25	40
40	30	60
35	30	18

```
for (int row = 0; row < marks.length; row++) {
for (int col = 0; col < marks [row]. length; col++) {
    System.Out.Print (marks [row] [col] + " ");
}
System.Out.Println ();
```

Multidimensional array → each with diff length.

```
int [] [] marks = new int [3] [];  //rows = 3
marks [0] = new int [1];
marks [1] = new int [2];
marks [2] = new int [3];
marks [0] [0] = 23;
marks [1] [0] = 24;
marks [1] [1] = 90;
marks [2] [0] = 12;
marks [2] [1] = 78;
marks [2] [2] = 45;
for (int row = 0; row < marks. length; row++) {
  for (int col = 0; col < marks [row]. length; col++) {
    System.out.Print (marks [row] [col] + " ");
  }
System.Out.Println ();
```

2D Arrays.

```
int [ ] [ ] marks = {
 { 12, 14, 56 },
 { 34, 45, 67 },
 { 45, 67, 78 }
};
 for (int row = 0; row < marks.length; row++) {
  for (int col = 0; col < marks[row].length; col++) {
    System.Out.Print(marks[row][col] + " ");
  }
  System.out.println ();
}
```

Strings in Java.

```
String fname = "Aditya";
String lname = "Tandan";
System.out.Println (fname+lname);
String fullname = fname + " " + lname;
```

Aditya Tandan

A + " " + B

How Arrays work in Java, Heap. Contiguous Storage, Random Access. Java.

Day 10

Arrays. `int [] arr = new int [5];`

arr

5	4	6	2	3
0	1	2	3	4

Datatypes.

System.out.Println (arr[2])

Random Access.

Premitiw	Nonpremitiw
integers	Arrays.
floating	String
Boolean	Object
Char	

int x = 4

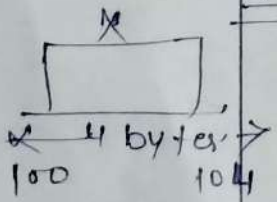
Means 'x' directly hold the value 4.

Stack. Memory

Premitiw.

int = 4 bytes.

x = 4; [4]



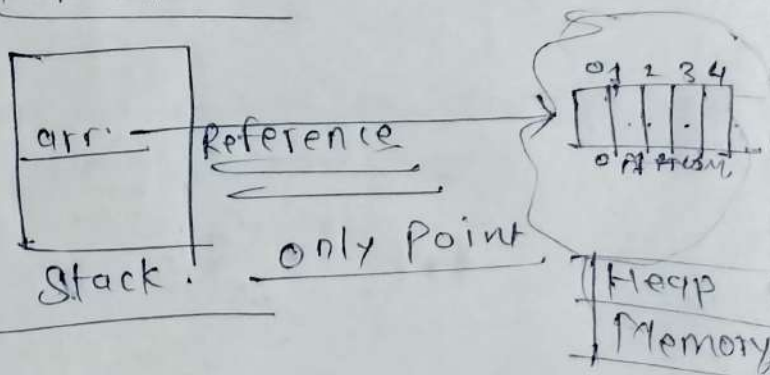
System.out.Println, 100 – 104

Non Premitiw

int

int [] arr = new int [5];

Nonpremitiw



arr it does not directly holds the array.

⇒ it is a reference to an array.

10	20	30	40	50
0	1	2	3	4
100	104	108	1012	1016

1020

arr [0] = 10 → 100

arr [1] = 20 → 104

arr [2] = 30 → 108

→ 1012

50 → 1016

System.out.Println (arr[3]);

arr [3] = 100 + (4 × 3) = 1012

arr [4] = 100 + (4 × 4) = 116.

arr[i] = Base Address · (D.T. size · x − i) = kisi bhi random index

System.out (arr[3]);
100 + (4 × 3) = 112

int [] arr = new int [5];
long → 8 B, float 4 B, Double 8 B.

Boolean → true, false, char 2 B →
According to Efficient java Docs.
No fix size. oracle, open jdk, Hotspot

→ Boolean → 1 Bit

T	f
1	0

byte = 8 bit
1 bit

for Better CPU Optimization 1 byte

Boolean [] arr = new boolean [5]

f	T	f	T	f
0	1	2	3	4

100, 101, 102, 103, 104, 105

arr [2] = 100 + (1 × 2)
= 102 — false

formost of the JVM CPU optimization

arr

0	1	2	3	4

100, 101, . . . 204, 208

arr [100]; 100 + (4 × 100)
= 204

if (index < 0 || index > arr.length)
throw Array index out of Bound.;
}.

Array of Arr

Exceptional Handling | Array index out of bound Exception.

2D Arrays. int [] [] arr = new int [3] [4];

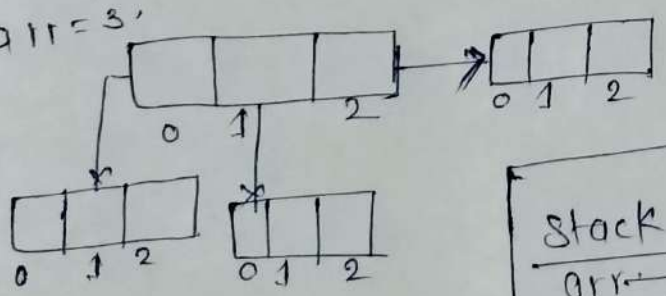
We Have of size 3, where each element is itself an array of size = 4.

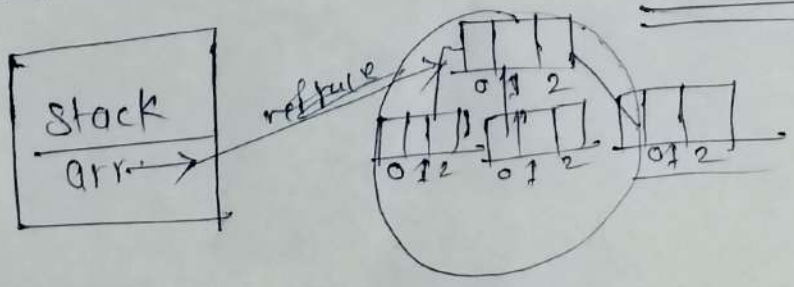
arr [1], [2]

	0	1	2	3
0	2	3	4	5
1	6	3	9	8
2	9	2	6	7

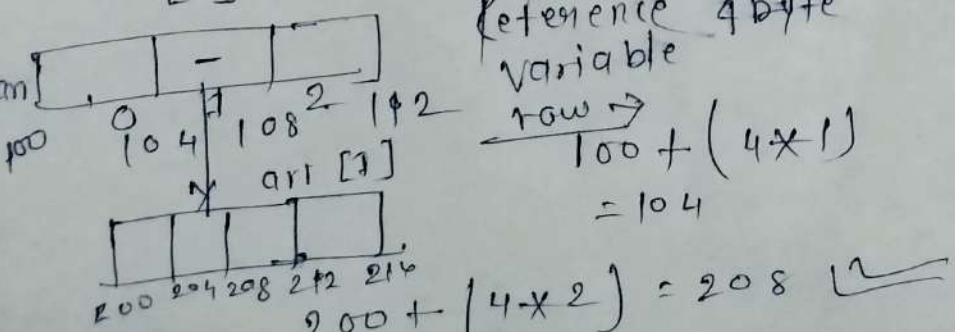
arr [1] [2]

arr = 3,





arr [1] [2].

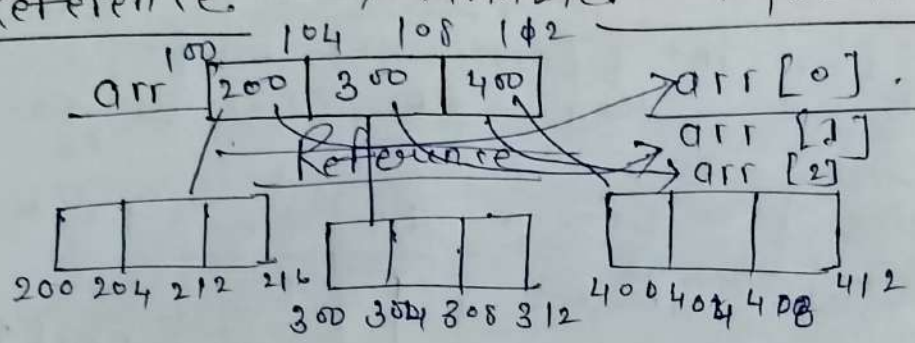


Reference 4 byte variable

row → 100 + (4 × 1)
= 104

200 + (4 × 2) = 208

Reference → Variable → location



arr[i][j]

row

Base Address + (4 × i) = 2

Base Address 2nd Array

col

[2 + (4 × j)].

String → Non Premitive data → 2D Array make.

String [ ] names = new String [3];

names → "Sachin" | "Ajay" | "Raj"
0 1 2

, string → ??
→ names [1];

names

0 1 2

"Aditya" "Rohit" "Ajay"
Ajay

Base Address - upto jdk9. string change nota this

100 | 200 | 300

0 1 2

400 404 408 "Ajay"

400 + (4 × 1) = 404

Random Access → arr[3]; →

0 1 2 3

casting = Byte use